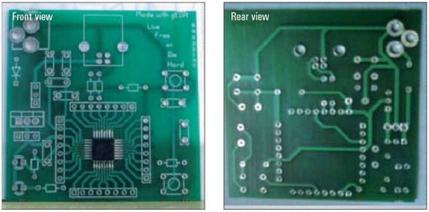

Figure 5: Machine-fabricated board designed by author (AT90USB162 mini development board)

BOM (bill of materials) generation and DRC (design rule checking) can also be outputted from gnetlist.

For designing PCBs, the schematic file has to be converted to the *pcb*-readable file format. You can do this using *gnetlist*, as mentioned. However, this is a slightly complicated process. So one of the gEDA hackers made a tool called *gsch2pcb* in C. This does the schematic to *pcb* file format conversion, automatically.

PCB, the layout program

The *pcb* tool helps you draw the wires for a custom printed circuit board (PCB). *pcb* is, in fact, a specialised drawing program to draw metal tracks, components, drill holes and other structures onto your circuit board. Its user interface presents a drawing window accompanied by all the widgets and tools necessary to draw your circuit board.

Designing printed circuit boards using *pcb* is very simple. Place the component footprints on the board, and route the tracks between the pins of the footprint. That's all! You have your PCB ready for fabrication. On the other hand, if you have drawn the schematic using *gschem*, you can use the command *gsch2pcb* to convert the schematic file into a *pcb*-readable format.

pcb supports around 16 copper layers, which, I'd say, is quite sufficient for most designs. Also, there is no limitation in the design size, number of components, PCB layer count, etc, which is an important advantage when compared to other proprietary EDA tools. Other features of *pcb* include a change in track width, importing logos on to the board, auto-routing, etc.

A complete design

Once the PCB layout is ready, you can export it into a wide variety of formats. If you need to fabricate a home-made PCB, you can take a laser printout and use a hot iron to transfer the PCB pattern to the copper clad board. This method of transferring the PCB layout from a sheet of paper to copper cladding is nicely described online—Google for more information! If you don't have a printer at home, then export the file to the *ps* format and then later to a PDF. Figure 4 shows a home-made PCB that I fabricated.

However, some of the designs might be complex -- the ones for SMD components, for instance. These types of boards are hard to fabricate at home. Under the circumstances, first export the PCB file into a Gerber file format. Then send the Gerber file to a PCB fabricator near your locality. The PCB fabricator uses machines for fabricating boards. The details for the PCB fabricating machine are in the Gerber files. This kind of fabrication generally produces a high-quality board as compared to a home-made PCB, but is much costlier. Figure 5 shows a board fabricated by sending a Gerber file to the PCB fabricator. (Figure 3 shows a PCB design that's made using gEDA.) Compare the quality of the home-made and machine-fabricated PCBs.

There are many big open source hardware projects that use gEDA for designing. Some of them include Ronja (Reasonable Optical Near Joint Access), Darrell Harmon's Single Board Computer Project, MINT, etc. There are some projects contributed by the author also.

Other tools and design flows

Remember that gEDA itself is a collection of tools. It contains a wide variety of tools for electronic design, and describing each will take up a lot more pages. And yet, gEDA is not the ultimate in electronics engineering tools available for GNU/Linux. There are still a lot more companion projects. Some of these, along with their applications, are described below:

- Gerbv– Gerber viewer
- Gnucap – Next-generation analogue circuit simulation
- Ngspice – SPICE3f5 (analogue simulator)
- Icarus Verilog – Verilog compiler and simulator
- GTKWave – Waveform viewer
- Wcalc– Transmission line and electromagnetic structure analysis END

Links and references

- Official gEDA website: *www.gpleda.org*
- gEDA Wiki Page: *en.wikipedia.org/wiki/GEDA*
- gEDA tutorials and projects: *www.delorie.com/pcb*
- gEDA symbol library: *www.gedasymbols.org*

By Jeffrey Antony

The author is a 21-year-old electronics engineer from Kerala, currently enrolled in a PG Diploma in embedded systems at the DOEACC Centre, Calicut. He is a free software and open hardware enthusiast. In his free time he likes to go cycling, listen to music, and work on open hardware projects. You can find more about the author at www.jeffrey.co.in